Automated PCB Tester **Avani Yadav** **Ryan Qi**

Our project prototypes an automated manufacturing line machine for large-scale PCB validation. In our current design, the user captures an image and selects a pad to probe (Stage 1). The system acutates motors to position the probe (Stage 2), then analyzes and visualizes the probed signal (Stage 3).

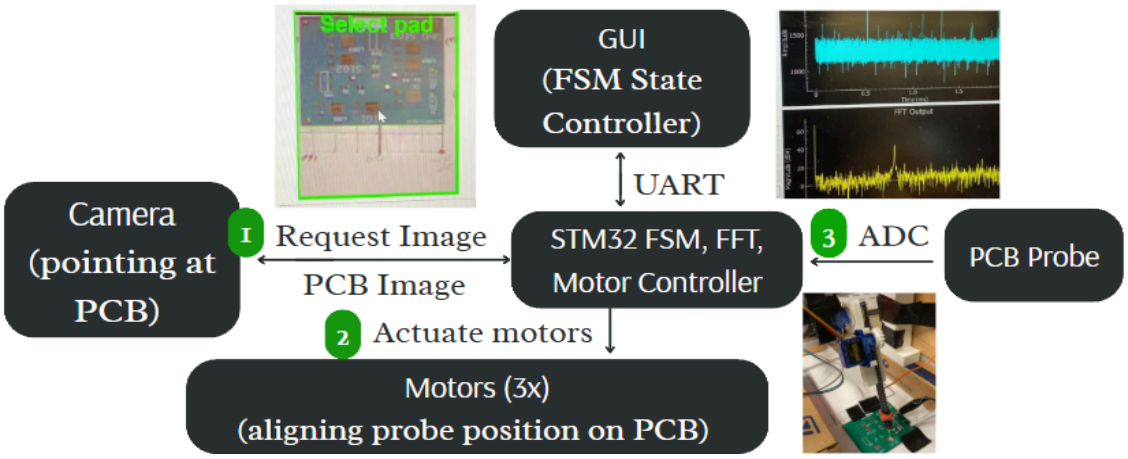

Figure 1: Block Diagram of the Automated PCB Tester

Bandwidth Analysis

Operation	Description	Min. Time
1) Camera image transfer (115,200 baud 8N1 UART)		
A: UART commands	Used for GUI to change the FSM state with a 10-byte fixed command echoed by the STM32 as a software handshake. (86.8 us per frame)	173.6 µs
B: DCMI	DCMI runs at 168 MHz, max data rate: 1503360 bytes/sec	0.0746 ms
C: Transfer image to GUI	174x144, 16 bit pixels over UART. 50112 bytes per frame.	4.35 s
2) Motors - two I²C controlled Servos w/ PCA9685 driver, Stepper w/ GPIO on L298 driver		
D: Stepper Motor GPIO control	L298N driven using HAL_GPIO_WritePin with 100 Hz square waves & HAL_Delay. GPIO toggling takes ~50 ns (6–8 instr. at 168 MHz, -O3, with fetch and bus overhead), L298N propagation delay is ~4 µs. [1]	4.05 µs
E: Stepper move	Experimentally, at 1 A, 100 Hz input, speed = 3 cm/s. Typically, this equates to 1-2 s of movement time, bottlenecking the system.	1.5 s (typ.)
F: Servo I²C control	Writing PWM to driver takes 6 bytes (~600 µs over I²C). 25 MHz driver clock gives negligible delays making I²C the bottleneck [2]	~600 µs
G: Servo move	0.1 s/60° (5V). 0.3s to rotate full 180°	~300 ms
3) ADC & FFT - Signal Analysis		
H: Timer triggered ADC	2048 samples, 21 MHz ADC with DMA, triggered by TIM2 at 1 MHz. With 3 cycles per sample, our ADC sampled at 1 MSps.	2.05 ms

I: CMSIS M4 FFT*	2048 sample float format (with FPU): 168 MHz, based on CMSIS benchmarks, -O3 optimization [3].	650–750 µs [3]
J: Transfer	FFt 2048 * 4 bytes and ADC is 2048 * 2 bytes, UART at 115.2 kbaud, 8N1.	1.07 s

**Green = full power operation, yellow = eligibility for low-power modes (LPMs).*

The greatest power saving potential is through using LPMs during non-CPU tasks[1]. For example, GPIO, ADC, I²C, UART can all be done while in SLEEP mode with the aid of DMA, timers, interrupts. Also, STOP mode can be used during idle time like human/GUI delay.

Energy savings are based on hypothetical operation with/without LPMs, where the user (with 0.5 s delay) captures an image, sets a sequence of probing 50 pads, and the system autonomously copies this sequence for 1000 subsequent PCBs, generating a validation report at the end.

Energy consumed[2] *= [Y+A+B+C+(Y+A+D+E+F+G+Y+H+I+J)*N+(A+D+E+F+G+H+I+J)*N*(X-1)]*

Energy Saved = ***27.8 kJ - 5.5 kJ => ~80% less energy using LPMs***

Code breakdown

Module	Resources	Use of Resources
Camera	Lab 5 ov7670.h and ov7670.c	Reused code, added colour feature independently
Servo	PCA9685 interface [4]	Modified the “PCA9685” functions to our use case
Stepper	N/A	Independently written
ADC+FFT	ARM_CMSIS_FFT [5]	Used FFT function and documentation
FSM	UART RX/TX [6]	Adapted into function pointer FSM state transition logic

Project Reflection

Initially, our goal was to automatically detect PCB pads via image processing. With time constraints, we opted for user-selected probing through the GUI. The system would hypothetically operate with the same sequence on thousands of identical PCBs, so having the user “program” this one manual sequence was justified. A redo of this project might involve more focus on electrical tests, automation, and showcasing best-case probing speed.

High-resolution PCB images and signal data reveal proprietary layouts and functionality. Without proper safeguards, this could lead to IP exposure or misuse. To address these risks, test case reports would be encrypted, with data storage only after proper, clear authorization provided. Additionally, more automated versions of this design could result in job replacement. However, such tools can be designed to enhance engineer productivity by automating repetitive tasks. An ethical design of our product would prioritize human-in-the-loop systems to support, not replace, skilled labour.

Ryan’s work: Stepper motors, ADC, FFT, UART (handshakes), GUI
Avani’s work: Servo motors, Camera calibration, UART (RX/TX configuration)
Combined work: Colored images, FSM, Motor calibration, Mechanical CAD & BOM

---

[1] RUN draws 56.75 mA = 187.3 mW, SLEEP draws 12.9 mA = 42.6 mW, STOP draws 55 uA = 181.5 uW [4]
[2] Assume Y = STOP energy (idle time (0.5) * 3.3 * STOP current), N = 50 pads probed, X = 1000 PCBs, {A,B,C…H,I,J} = Power consumption of operation mode * Min. time (from Bandwidth Analysis)

References

[1] STMicroelectronics, “L298 – Dual Full Bridge Driver,” Product Page, Oct. 2023. [Online]. Available: https://www.st.com/en/motor-drivers/l298.html. [Accessed: Apr. 6, 2025].

[2] NXP Semiconductors, “PCA9685 – 16-channel, 12-bit PWM Fm+ I²C-bus LED controller,” Product Data Sheet, Apr. 2015. [Online]. Available: https://cdn-shop.adafruit.com/datasheets/PCA9685.pdf. [Accessed: Apr. 6, 2025].

[3] STMicroelectronics, “Digital signal processing for STM32 microcontrollers using CMSIS,” Application Note AN4841, Feb. 2018. [Online]. Available: https://www.st.com/resource/en/application_note/an4841-digital-signal-processing-for-stm32-microcontrollers-using-cmsis-stmicroelectronics.pdf. [Accessed: Apr. 6, 2025].

[4] MicroPeta, “PCA9685 Interface Reference,” MicroPeta Video Tutorial, [Online]. Available: https://www.micropeta.com/video113. [Accessed: Apr. 6, 2025].

[5] ARM, “CMSIS DSP Software Library,” GitHub Repository, [Online]. Available: https://github.com/ARM-software/CMSIS_4. [Accessed: Apr. 6, 2025].

[6] STMicroelectronics Community, “Implementing UART receive and transmit functions on an STM32,” ST Community Knowledge Base, [Online]. Available: https://community.st.com/t5/stm32-mcus/implementing-uart-receive-and-transmit-functions-on-an-stm32/ta-p/694926. [Accessed: Apr. 6, 2025].

[7] STMicroelectronics, “STM32CubeMX Power Consumption Calculator,” STM32CubeMX Software Tool. [Accessed: Apr. 6, 2025].